Companion formal y computacional del Paper V Preprint 1.0

Juan Pablo Traverso Gianini

18 de octubre de 2026

Este documento describe el desarrollo formal final y el material computacional que acompañan a PAPER_V_preprint_v1.0_en.md y PAPER_V_preprint_v1.0_es.md. El Paper V estudia empaquetamientos finitos y realizaciones físicas generales. El Paper IV demuestra la cota para todo orden salvo una constante aditiva absoluta y el máximo exacto eventual descritos en el artículo principal. La aplicación IV–V separada no es una premisa de los resultados generales de V.

1. Fuentes canónicas e interfaz pública

El punto de entrada canónico es `PaperV.lean`. Su clausura de importaciones públicas contiene 122 módulos locales. La inclusión de archivos, su compilación, su importación pública y la disponibilidad de los teoremas se comprueban por separado: el build agregado alcanza los módulos de demostración, los gates de enunciados emiten tipos exactos y dependencias axiomáticas, y la auditoría completa recorre las declaraciones de esa misma clausura. El paquete también contiene gates independientes; son programas de validación, no hipótesis adicionales de los teoremas públicos.

El entorno es `leanprover/lean4:v4.35.0`, con Mathlib fijada en 56a005855ea17e0b0958f73ec801ae140b0a1679. El archivo de fuentes es PAPER_V_preprint_v1.0_FORMAL_SOURCE_435.zip; su resumen criptográfico exacto y el manifiesto por archivo están registrados en FREEZE_VALIDATION.json. Las cabeceras reconocen a Juan Pablo Traverso Gianini por la formalización e integración y conservan las atribuciones matemáticas y los créditos de fuentes y adaptación existentes. No reasignan la autoría de los resultados clásicos.

Comprobación	Alcance final
Raíz pública	`PaperV`
Build agregado	Código 0; 2582 tareas
Auditoría completa de declaraciones	Código 0; 2838 declaraciones, 122 módulos, 5547 visitadas
Gates de tipos exactos y axiomas	10 gates; 403 consultas de tipos y 431 consultas de axiomas
Axiomas no estándar	Cero
Unión de axiomas fundacionales	`propext`, `Classical.choice`, `Quot.sound`
Correspondencias principales y de interfaces auxiliares	37 y 11

El archivo de fuentes contiene las fuentes locales de Lean comprobadas, la configuración del paquete, los textos legales y un manifiesto relativo por archivo. El directorio separado 02_validation/PREPRINT_1_0/ contiene los logs finales, los recibos de salida y la clausura pública exacta. Antes de recorrer las declaraciones, la auditoría comprueba que todos los módulos enumerados

estén presentes en el entorno importado. Las atribuciones matemáticas y los créditos de las fuentes se conservan en el desarrollo canónico.

2. Arquitectura e hipótesis

Los módulos de sistemas de conflicto y copias acotadas definen conjuntos factibles, defectos de colorabilidad y desigualdades de transferencia. Los módulos de certificados de recursos conectan relajaciones de capacidad reales y certificados primal–dual con multiconjuntos de ocurrencias. Los módulos de realizadores y anfitriones convierten una clase de color seleccionada en un empaquetamiento físico; el argumento no toma la unión de clases factibles independientes. Los módulos de copias, perfiles puros y promoción mixta controlan los recursos físicos compartidos. Los módulos de frontera unitaria proporcionan restricción, completación y transporte dual. La sensibilidad finita y la contabilidad en árboles finitos conservan su propio alcance explícito.

Varias hipótesis deben quedar visibles junto al mapa de declaraciones. El parámetro de multiplicidad satisface $q \geq 1$. El objetivo ponderado y el número de clases se fijan antes de seleccionar un certificado. El LP finito acotado de recursos usa conjuntos de recursos no vacíos donde se requieren; la relajación general permite conjuntos de recursos vacíos. Las familias de admisibilidad usadas para tomar un mínimo son no vacías. Un realizador físico debe cumplir las condiciones reales de disjunción de recursos. Puentes demostrados relacionan las entradas y los certificados racionales con los óptimos reales. Los resultados de copias y copias mixtas conservan sus hipótesis de multiplicidad y recursos compartidos. Los resultados genéricos de sensibilidad no identifican una familia WSNF no demostrada; las cotas de árboles finitos no son cotas globales asintóticas de pérdida.

2.1. Cobertura de los resultados de apoyo

Elemento	Resultado comprobado y límite
F1	Programa lineal finito de recursos efectivo, certificados racionales primal/dual, realizaciones físicas de clases y cota con dos errores, incluidos puentes entre óptimos racionales y reales
F2	Portafolio finito de certificados de coste mínimo para un objetivo y número de clases fijos; uniones físicas compatibles con recursos disjuntos
F3	Asignaciones voraces de anfitriones por suma de grados, validez física de recursos, pesos racionales heredados y evitación de un empaquetamiento de fondo fijo
F4	Promedio ponderado de factores emparejados y completación física de los supervivientes, incluido el caso de base vacía
F5	Restricción/extensión exacta de frontera unitaria, levantamiento dual explícito, fórmulas de óptimos enteros y fraccionales y conservación de la brecha; ganancias originales no negativas, ganancias centrales reducidas posiblemente con signo
F6	Promoción física mixta de triángulos/K4, conservación de capacidades, identidades de valor y una secuencia acotada de normalización que conserva la racionalidad
F7	Sensibilidad genérica de politopos finitos, continuidad, extensión a familias densas y estabilidad de brechas finitas; no se afirma ninguna identificación WSNF ni clasificación no demostrada
F8	La contabilidad comprobada de árboles finitos conserva su holgura fraccional y sus pesos exactos; no es una estimación global de pérdida asintótica

3. SEAL: implementación, garantías y certificados finitos

Los resultados matemáticos permanecen en la sección 8 del artículo principal. La Proposición 8.1 establece la factibilidad de las operaciones especificadas, la Proposición 8.2 acota las reparaciones de mejora aceptadas y el Teorema 8.3 proporciona los regímenes de anfitriones exactos o

garantizados allí indicados. Sus declaraciones formales aparecen en la sección 5 de este documento. Lean verifica esas interfaces matemáticas bajo sus hipótesis; no verifica la implementación Python, JavaScript o C++ de extremo a extremo ni su tiempo polinomial.

3.1. Implementación y modelos actuales

El software se distribuye en 03_reproducibility/SEAL/. Su directorio `current/` contiene las rutinas Python BASE, el programa v3 y los puertos JavaScript/C++ de BASE. `replay/` contiene entradas de prueba completas, salidas guardadas, verificadores exactos y tablas de resultados actuales. El manifiesto fija los bytes exactos de las fuentes descritas aquí.

Las variantes BASE usan cardinalidad estándar, pesos no negativos de pares, capacidades de grado o un potencial registrado, según el modelo de entrada especificado. El programa v3 se restringe a pesos unitarios y capacidad uno. Se rechazan las llamadas v3 ponderadas o de capacidad superior no admitidas; las variantes BASE correspondientes siguen siendo interfaces separadas. Los puertos exponen las familias físicas seleccionadas además de sus valores objetivos, lo que permite comprobar independientemente la factibilidad en cada lenguaje ensayado.

La operación N1 actual es transaccional. Libera provisionalmente una asignación que bloquea, comprueba su relocalización en otro anfitrión compatible y después comprueba la inserción del par descubierto en el anfitrión original. Sólo confirma un movimiento factible con ganancia objetiva estrictamente positiva. Si falla, restaura el estado anterior completo. Por ello, deben especificarse el objetivo y las restricciones de capacidad aplicables antes de interpretar una ejecución.

```
R := a feasible host assignment
for an uncovered pair e and a compatible host j:
    choose a blocking assignment (f, j)
    save the current state and release (f, j)
    try relocating f to another compatible host
    try inserting e into j
    commit only if feasible and the objective strictly increases
    otherwise restore the saved state
```

Las rutinas BASE y de respaldo de v3 implementan N1. La rutina separada `reference_v1/` implementa N1 y N2 para el modelo sin pesos y de capacidad uno; no implementa todas las variantes BASE. En esa referencia, la selección por escasez va seguida de una búsqueda exhaustiva en los entornos finitos, que se reinicia tras aceptar una mejora. La auditoría LP opcional se ejecuta después de la construcción.

La rama bipartita de v3 valida la asignación propia desde listas que obtiene antes de utilizarla. No se afirma que el ejecutable construya una asignación en todo el rango del teorema BKW. Una asignación individual satisfactoria o la coincidencia de cotas inferior y superior proporciona su propio certificado finito. Los regímenes matemáticos especiales exactos no implican una garantía general de aproximación para la búsqueda local.

3.2. Resultados finitos actuales

Comprobación del conjunto actual	Resultado verificado
Empaquetamientos físicos Python	110: 72 BASE y 38 v3
Empaquetamientos físicos JavaScript y C++	54 en cada lenguaje; todos los valores objetivos coinciden con Python
Duales racionales completos de cardinalidad	38; 149,819 desigualdades de objetos y 47,015 filas comprobadas
Óptimos de cardinalidad certificados por cotas iguales	12: seis casos de distribución, dos testigos y cuatro casos mayores
Intervalos certificados positivos	26: doce gaps de distribución de 1–3 y catorce gaps de casos mayores de 1–40
Controles negativos independientes	Siete rechazados

Las pruebas BASE usan dieciocho entradas guardadas en cada uno de cuatro modos. Las entradas ponderadas usan pesos en `{1,2,5,10}`; las de mayor capacidad asignan capacidad dos a cada anfitrión. Cada puerto comprueba los modos estándar, ponderado y de capacidad dos sobre esas entradas. Las pruebas v3 usan pesos unitarios y capacidad uno: dieciocho instancias de distribución, dos testigos bipartitos y dieciocho configuraciones grandes guardadas. Las comprobaciones de factibilidad y objetivo ponderadas y de capacidad dos no proporcionan certificados de optimalidad para esos modelos. Los valores actuales completos por instancia se registran en replay/CURRENT_RESULTS.csv.

Un testigo físico N1 explícito pasa de un par seleccionado a dos. La regresión transaccional finita adicional comprueba 685 estados iniciales factibles y 27,708 intentos de operación, con 3,180 inserciones aceptadas y 24,528 intentos rechazados que restauran el estado. Esas cifras corresponden a dos sistemas de anfitriones especificados, capacidades uno/dos y ganancias cero/uno; no son afirmaciones universales de corrección.

El verificador de reproducción reconstruye la pertenencia a los anfitriones, la unicidad global de pares, las capacidades de grado por anfitrión, las cardinalidades y los objetivos ponderados desde las especificaciones completas de entrada. Las pruebas transaccionales comprueban mejoras N1 aceptadas, la restauración tras intentos fallidos, el comportamiento con peso cero y los modelos de capacidad indicados. Son comprobaciones finitas ejecutadas, no demostraciones universales de corrección del programa.

Para cada dual racional guardado, reconstruye todos los objetos y filas de restricciones, comprueba los precios no negativos y la cobertura, y recalcula exactamente el objetivo. Un empaquetamiento factible da una cota inferior; la parte entera de una cota superior racional válida da una cota superior entera. La coincidencia de cotas certifica el óptimo de esa salida individual, mientras que un intervalo positivo registra incertidumbre. No se usa como certificado un mero estado del solver de punto flotante ni un objetivo numérico.

La comparación entre los tres lenguajes se limita a los modelos de entrada y las salidas guardadas. No demuestra equivalencia general, y los tiempos no son premisas matemáticas. El informe de resultados distribuido con el software conserva los certificados exactos actuales, los hashes de fuentes y los límites de alcance.

El conjunto adicional de referencia de capacidad uno contiene treinta instancias guardadas y 147,111 objetos anfitrión–par. Catorce intervalos certifican óptimos enteros; dieciséis conservan incertidumbre positiva, de ancho entero a lo sumo 37. Seis óptimos pequeños tienen enumeración independiente. Se rechazan cuatro certificados alterados y dos pruebas explícitas ejercitan N1 y N2. Doce casos compactos habilitan reparaciones; dieciocho casos mayores sólo ejecutan selección y auditoría dual. Las restricciones de conjuntos impares se generan para paletas de ta-

maño a lo sumo nueve; omitir conjuntos impares mayores puede debilitar una cota sin invalidar un dual comprobado. Este conjunto no implica exactitud universal de la relajación.

3.3. Comprobaciones de certificados escalares

Certo 0.34.0 comprueba cinco consecuencias escalares codificadas. Hacen explícitas las hipótesis aritméticas y proporcionan comprobaciones reproducibles; no demuestran la existencia de un grafo, un realizador físico ni un empaquetamiento factible.

Obligación	Papel matemático	Condición de reproducción
two_error_transfer	Combinar pérdidas de relajación y realización	Requiere SMT
integer_dual_stop	Parar cuando coinciden las cotas inferior entera y superior dual	Requiere SMT
composed_realization_errors	Sumar los presupuestos de error de realización indicados	Requiere SMT
improving_repairs_budget	Acotar las reparaciones de mejora aceptadas	Certificado sin solver
gap_residual_and_resource_defect	Separar residuo y defecto de recurso	Requiere SMT

Un sexto certificado, independiente del solver, es un control negativo de una implicación escalar deliberadamente insuficiente: $\nu = \mathrm{cov} = 0$ y un valor de relajación $1/2$ satisfacen esas premisas codificadas y dejan un gap de relajación positivo. No es un contraejemplo de grafos ni se afirma que proceda de un sistema de anfitriones. En particular, no contradice las cotas superiores de recursos conservadas por el modelo real de SEAL. Cada comprobación vincula el hash de la especificación y la obligación; se rechaza la comprobación contra una obligación diferente. `RESULTS.json` registra qué comprobaciones requieren un solver SMT.

4. Reproducción

Extraer PAPER_V_preprint_v1.0_FORMAL_SOURCE_435.zip y ejecutar desde la raíz del paquete con las dependencias fijadas:

```
lake build PaperV
lake env lean PaperVManuscriptGate.lean
lake env lean PaperVSupplementGate.lean
lake env lean PaperVImprovementsGate.lean
lake env lean PaperVReleaseGate.lean
lake env lean FreezeFullAudit.lean
```

CHECKED_PUBLICATION_GATES_435.json registra los diez gates, los comandos exactos, las declaraciones consultadas y sus dependencias axiomáticas. MANUSCRIPT_MAP.json y SUPPLEMENT_MAP.json dan las correspondencias exactas por enunciado. SOURCE_CLOSURE.json identifica los módulos públicos alcanzados por el agregado. La auditoría no admite nuevos axiomas matemáticos.

Desde 03_reproducibility/SEAL/, comprobar las salidas guardadas con la biblioteca estándar de Python:

```
python replay/scripts/check_outputs.py --source-root current
```

Para regenerar el conjunto actual, usar las dependencias y entornos de ejecución fijados en su README y ejecutar:

```
python replay/scripts/run_suite.py --source-root current
python replay/scripts/run_ports.py --source-root current
python replay/scripts/check_outputs.py --source-root current
```

Los testigos adicionales de referencia y los certificados escalares tienen comandos separados:

```
python reference_v1/check_certificate.py
python certo_v/run_certo.py
```

Ejecutar Python normalmente, sin `-O`, para mantener las aserciones. Comprobar los testigos guardados no requiere el solver numérico que generó sus precios. La generación y las comprobaciones de Certo que necesitan SMT requieren sus dependencias documentadas; generar los puertos requiere Node y un compilador C++17, que puede especificarse con `--cxx`. El manifiesto del software vincula fuentes, entradas, vectores completos, salidas y registros de validación.

5. Mapa de declaraciones por enunciado

Las 37 correspondencias siguientes identifican las declaraciones públicas. Los tipos exactos, las hipótesis y las conclusiones separadas están en las fuentes y en la salida de los gates; este índice no suprime las precisiones de la sección 2.

Elemento del artículo principal	Declaraciones canónicas
Definición 2.1	PaperV.ConflictSystem; `PaperV.CSFeasible`; `PaperV.nu`
Definición 2.2	`PaperV.Copies`; `PaperV.Hgraph`; PaperV.IsQMultiset
Definición 2.3	`PaperV.Dq`; `PaperV.oct`; PaperV.Dq_one_eq
Definición 2.4	PaperV.Manuscript.ResourceRelaxation; PaperV.ResourceCertificate.System; PaperV.ResourceCertificate.System.incidence
Definición 2.5	PaperV.Manuscript.ResourceRelaxation; PaperV.Manuscript.ResourceRelaxation.Optimum
Lema 3.1	`PaperV.splitting`
Corolario 3.2	PaperV.nu_ge_ceil
Corolario 3.2a	PaperV.cover_witness_defect; PaperV.triangle_cover_defect
Corolario 3.2b	PaperV.Manuscript.minimum_coloring_rigidity_iff
Teorema 3.3	PaperV.Manuscript.resource_half_integral_transfer
Teorema 3.4	PaperV.Manuscript.resource_integral_transfer
Teorema 3.4w	PaperV.transfer_q_weighted
Lema 3.5	PaperV.parity_localization
Definición 4.1	PaperV.ClassRealizer; `PaperV.RealCert`; PaperV.RealCert.cost
Teorema 4.2	PaperV.realizable_class_transfer; PaperV.realizable_transfer_cert
Definición 4.3	`PaperV.wRDq`
Teorema 4.4	PaperV.wRDq_identity_eq_wDq; PaperV.wRDq_le_wDq
Ejemplo 4.5	PaperV.StarExample.intermediate_defect
Corolario 4.6	PaperV.ResourceCertificate.two_error_real_optimum; PaperV.ResourceCertificate.resource_gap_identity; PaperV.ResourceCertificate.exists_real_optimum
Proposición 5.1	PaperV.Host.csFeasible_hostCS_of_properOn
Teorema 5.2	PaperV.Galvin.galvin_multigraph_max_degree; PaperV.Galvin.galvin_bkw_local
Corolario 5.3	PaperV.Host.galvin_host_completion_exact; PaperV.Host.bkw_host_completion_exact; PaperV.Host.galvin_lifted_transfer; PaperV.Host.galvin_local_lifted_transfer
Ejemplo 5.5	PaperV.Strictness.strict_separation; PaperV.Strictness.nu_hostCS_kr
Afirmación auxiliar 6.1a	PaperV.copy_integral_edgeLoad_le_two
Teorema 6.1	PaperV.copy_fractional

Elemento del artículo principal	Declaraciones canónicas
Teorema 6.2	PaperV.copy_integral_for_optimal_packings; PaperV.copy_integral_for_packings; PaperV.triangleImageMultiset
Lema 6.2k	PaperV.third_subset_union; PaperV.k4_of_three_triangles; PaperV.triCS_clique_card_le_four
Corolario 6.2a	PaperV.copy_triple_bound; `PaperV.tight_oct`
Proposición 7.1	PaperV.PureProfile.prop43_nu; PaperV.PureProfile.prop43_muMat
Proposición 7.1a	PaperV.PureProfile.nu_ge_frac_muMatDeg; PaperV.PureProfile.nu_ge_two_thirds_muMatDeg; PaperV.PureProfile.nu_ge_muMatDeg_two
Proposición 7.1b	PaperV.PureProfile.muMatDeg_eq_min; PaperV.PureProfile.nu_eq_muMatDeg_of_even; PaperV.PureProfile.nu_eq_frac_muMatDeg_of_odd
Teorema 7.3	PaperV.Mixed.mixed_copy_fractional; PaperV.mixed_copy_integral
Corolario 7.3a	PaperV.Mixed.mixed_copy_integral_of_roundingGap
Proposición 8.1	PaperV.feasible_run; PaperV.SealBridge.sealFeasible_of_properOn
Proposición 8.2	PaperV.accepted_repairs_le_cov; PaperV.accepted_repairs_le_audit
Teorema 8.3	PaperV.SealBridge.seal_galvin_regime; PaperV.SealBridge.seal_bkw_regime; PaperV.SealBridge.seal_konig_guarantee
Proposición 9.3	PaperV.Steiner.exists_quadratic_defect_frac

La interfaz del Teorema 6.2 recibe los dos empaquetamientos copiados factibles especificados y construye su multiconjunto imagen real; el adaptador para empaquetamientos óptimos conserva esas mismas familias. La interfaz del Teorema 7.3 también utiliza los datos reales de la copia mixta. Se conservan adaptadores existenciales por conveniencia, pero no sustituyen estas correspondencias.

6. Interfaces matemáticas auxiliares

Las once correspondencias siguientes localizan los resultados auxiliares en el proyecto de fuentes entregado. Sus etiquetas identifican el inventario matemático de las fuentes; los tipos exactos determinan todas las hipótesis. No hace falta un suplemento separado no distribuido para localizar estas interfaces.

Elemento conservado	Declaraciones canónicas
Proposición 3.6	PaperV.Dq_antitone; PaperV.Dq_le_oct; PaperV.Dq_mono_left
Proposición 3.7	PaperV.Dq_eq_zero_iff; PaperV.oct_eq_zero_iff; PaperV.Dq_eq_zero_iff_chromatic; PaperV.Dq_pos_iff_lt_chromatic
Proposición 3.8	PaperV.Dq_ge_card_sub; PaperV.defect_tight_all_max; PaperV.defect_eq_of_survivors; PaperV.Manuscript.minimum_coloring_rigidity_iff
Proposición 3.9	`PaperV.Dq_sum`; `PaperV.oct_sum`
Proposición 3.10	PaperV.Dq_lineGraph_eq_zero_iff; PaperV.Dq_lineGraph_pos_iff; PaperV.Dq_one_lineGraph; PaperV.profile_zero_set_not_values
Corolario 3.11	PaperV.TerminalPair.p4_133_existence_sharp; PaperV.TerminalPair.p4_133_existence_coarse
Teorema 7.4	PaperV.PairedFactors.exists_paired_completion
Proposición 7.5	PaperV.FiniteLP.gap_identity; PaperV.FiniteLP.weak_duality; PaperV.FiniteLP.complementarity_of_equal_value; PaperV.RationalFarkas.farkas; PaperV.RationalFarkas.not_sat_of_cert; PaperV.FiniteLP.exists_equal_primal_dual

Elemento conservado	Declaraciones canónicas
Corolario 5.4	PaperV.Host.weighted_host_selection; PaperV.Host.weighted_host_selection_nu; PaperV.Host.konig_weighted_host_selection; PaperV.Host.galvin_heavy_single_host
Lema C.1	PaperV.UnitBoundary.completion_optimal_unconditional; PaperV.UnitBoundary.unitBoundary_completion_optimal; PaperV.UnitBoundary.exists_optimal_normPack
Lema C.2	PaperV.UnitBoundary.exists_endpoint_repacking; PaperV.UnitBoundary.endpoint_repacking_card_bound; PaperV.UnitBoundary.endpoint_repacking_nonloss

El Lema C.1 tiene un adaptador incondicional de la fórmula del óptimo sin hipótesis de universo habitado ni de representantes; su versión con testigo normalizado conserva representantes explícitos. El Lema C.2 proporciona una familia real de representantes y un empaquetamiento físico, incluido el universo vacío; el enunciado de transporte conserva sus datos de empaquetamiento normalizado.

7. Alcance del registro final

El manuscrito principal contiene un párrafo de formalización; este companion reúne la arquitectura de fuentes, las correspondencias exactas, el informe computacional y los comandos de reproducción. La comprobación formal, la comprobación de certificados finitos, la prioridad bibliográfica y la revisión humana externa son obligaciones diferentes. La aplicación IV–V separada tiene su propio proyecto y registro de validación. El Paper V no importa la cota global de esa aplicación ni depende del Paper VI. Este companion acompaña el Preprint 1.0. La comprobación por el kernel y la verificación de certificados finitos no constituyen revisión humana externa ni certificación de prioridad histórica.

8. Reproducibilidad de los contraejemplos finitos

El directorio 03_reproducibility/counterexamples/ proporciona bk_completion_certificates.py y el certificado BK completo guardado. Desde ese directorio se ejecuta `python bk_completion_certificates.py --verify-only`, con las aserciones activadas. El comprobador usa sólo la biblioteca estándar, valida el dual racional de valor 19/2, enumera los 70 empaquetamientos óptimos de nueve elementos y comprueba el parámetro residual beta=2. Se rechaza un control negativo que altera un precio dual. La búsqueda de certificados sin `--verify-only` utiliza SciPy de manera opcional; verificar el certificado guardado no lo requiere. El número exacto 70 es un resultado computacional, no una declaración demostrada por el kernel de Lean. Declaraciones comprobadas por el kernel establecen por separado el óptimo BK y las cotas pertinentes, así como los hechos enunciados sobre CLON-2, STAR-T2 y Steiner.

9. Licencias y atribución

Copyright (c) 2026 Juan Pablo Traverso Gianini. Los materiales escritos se publican bajo CC BY-NC 4.0. Las contribuciones del autor a Lean y al software se publican bajo Apache 2.0; las dependencias y contribuciones de terceros conservan sus propias licencias y avisos. Las cabeceras de las fuentes y los textos legales adjuntos explicitan esta distinción. Se conservan las atribuciones matemáticas clásicas; reconocer una formalización no reatribuye el teorema subyacente.